COMMIT 10/13/2026 [AI Generated]

Compliance Engine — Backend Implementation Documentation

Project: Enterprise AI Governance

Module: Compliance Engine

Status: Backend foundation implemented and tested

Last updated: 13 October 2026

1. Project Overview

The Compliance Engine is a backend module of the Enterprise AI Governance project. Its long-term purpose is to evaluate company policies against regulatory requirements and identify compliance gaps.

The initial backend focuses on document ingestion, text extraction, OCR for scanned PDFs, and an API for initiating and retrieving document-processing results.

Current objectives

- Accept PDF documents through a REST API.
- Store uploaded documents locally.
- Extract text from digital PDFs.
- Recognize text in scanned PDFs using OCR.
- Initiate document processing through an analysis endpoint.
- Retrieve processing results using a generated analysis ID.
- Expose endpoints through Swagger UI for testing.

Important: The current analysis endpoint processes documents and returns extraction statistics. The actual regulation-versus-policy comparison engine has not been implemented yet.

2. Technology Stack

Technology	Purpose
Python 3.13	Backend programming language
FastAPI	REST API framework
Uvicorn	ASGI server
Pydantic	Request validation
CORSMiddleware	Cross-origin frontend access
`python-multipart`	Multipart file uploads
`pypdf`	PDF text extraction
PyMuPDF	PDF page rendering for OCR
Pillow	Image handling
`pytesseract`	Python interface to Tesseract OCR
Tesseract OCR	Text recognition from scanned documents
Swagger UI	Interactive API documentation and testing
React + Vite	Frontend application

Development environment

- Operating system: Windows
- Terminal: Git Bash
- Backend port: `8000`
- Frontend development port: `5173`

3. Backend Directory Structure

The current backend structure is organized as follows:

```
compliance-engine/
├── backend/
│   ├── main.py
│   ├── test_pdf.py
│   ├── uploads/
│   │   └── <document-uuid>.pdf
│   └── app/
│       ├── routers/
│       │   ├── health.py
│       │   ├── upload.py
│       │   └── analysis.py
│       └── services/
│           └── document_service.py
│
├── frontend/
└── README.md
```

The `uploads/` directory is located directly inside `backend/`. Uploaded PDFs are saved using generated UUID filenames.

The `test_pdf.py` script is used to verify text extraction independently of the API.

4. FastAPI Application Setup

File: `backend/main.py`

The main application initializes FastAPI, configures CORS, and registers the routers.

Core configuration:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.routers.health import router as health_router
from app.routers.upload import router as upload_router
from app.routers.analysis import router as analysis_router

app = FastAPI(
```

```
    title="Compliance Engine API",
    version="0.1.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "http://127.0.0.1:5173",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(health_router)
app.include_router(upload_router)
app.include_router(analysis_router)
```

CORS configuration

CORS allows the frontend development server to communicate with the FastAPI backend.

- Frontend: http://localhost:5173
- Backend: http://127.0.0.1:8000

These settings are intended for local development. Production deployment will require explicit production origins and appropriate security configuration.

5. Backend Startup

From the backend directory, start the server with:

```
python -m uvicorn main:app --reload --port 8000
```

The `--reload` option automatically restarts the development server when Python files change.

Available interfaces

Interface	URL
Swagger UI	http://127.0.0.1:8000/docs
ReDoc	http://127.0.0.1:8000/redoc
OpenAPI schema	http://127.0.0.1:8000/openapi.json
Health endpoint	http://127.0.0.1:8000/health/

Swagger UI is generated automatically by FastAPI; it does not need to be installed separately.

6. Health Endpoint

File: `backend/app/routers/health.py`

The health endpoint confirms that the backend is responding.

Endpoint

```
GET /health/
```

Verified response

```
{
  "status": "Backend Connected ✅",
  "project": "Compliance Engine",
  "version": "0.1.0"
}
```

The endpoint was tested successfully in the browser.

Purpose: Verify backend availability and provide basic project metadata.

7. PDF Upload API

File: `backend/app/routers/upload.py`

The upload endpoint accepts PDF files and saves them locally.

Endpoint

```
POST /upload/
```

Implementation features

- Accepts a PDF using multipart form data.
- Checks that the filename has a `.pdf` extension.
- Generates a UUID for each uploaded document.
- Saves the file in `backend/uploads/`.
- Limits file size to 10 MB.
- Returns the document ID, original filename, file size, and upload status.

Example response

```
{
  "document_id": "79f96287-7152-4bbe-954a-6bfd3a244cc8",
  "filename": "Anmeldung.pdf",
  "size_bytes": 1272335,
  "status": "uploaded"
}
```

The UUID shown is from the successful test upload.

Upload storage

Files are stored using their generated document IDs:

```
backend/uploads/<document-id>.pdf
```

Using UUIDs prevents files with identical original filenames from overwriting one another.

Verification

The endpoint was tested through Swagger UI and returned HTTP `200`.

Current limitations: Storage is local to the development environment. Extension checking alone does not guarantee that a file is a valid or safe PDF. Authentication, stronger file validation, and production storage controls remain future improvements.

8. PDF Text Extraction Service

File: `backend/app/services/document_service.py`

The document-processing service extracts text from uploaded PDF files.

Processing logic

1. Open the PDF.
2. Read each page.
3. Attempt normal text extraction using `pypdf`.
4. If a page contains very little extractable text, render the page as an image.
5. Run OCR on that image using Tesseract.
6. Collect the extracted text and page-level information.
7. Return processing statistics and extracted text.

Extraction libraries

pypdf

Used to extract embedded text from digital PDFs.

PyMuPDF

Used to render PDF pages into images for OCR.

Pillow

Used to load the rendered page image.

pytesseract and Tesseract OCR

Used to recognize text from scanned page images.

Output structure

The extraction service returns a dictionary containing:

```
{
    "page_count": 1,
    "character_count": 711,
    "ocr_pages": 1,
    "text": "...",
    "pages": [
        {
            "page": 1,
            "text": "..."
        }
    ]
}
```

The values shown are from the successful scanned-document test.

OCR fallback

The service uses OCR when normal PDF extraction returns fewer than 20 characters on a page.

This is a simple heuristic for the prototype. A more advanced implementation could evaluate text quality and support mixed pages containing both images and embedded text.

9. Tesseract OCR Installation and Configuration

Tesseract OCR is the external OCR engine. `pytesseract` is the Python interface used to call it.

Installation

Tesseract OCR was installed on Windows under the default directory:

```
C:\Program Files\Tesseract-OCR\
```

The executable is expected at:

```
C:\Program Files\Tesseract-OCR\tesseract.exe
```

Python configuration

The OCR service configures the executable path when necessary:

```
import pytesseract

pytesseract.pytesseract.tesseract_cmd = (
    r"C:\Program Files\Tesseract-OCR\tesseract.exe"
)
```

This path is specific to the current Windows environment. Other machines may need a different path.

Language configuration

The extraction service uses:

```
text = pytesseract.image_to_string(
    image,
    lang="eng+de"
).strip()
```

This requests English and German OCR models.

Both language models must be installed. The available models can be checked with:

```
"/c/Program Files/Tesseract-OCR/tesseract.exe" --list-langs
```

Version verification

The following command successfully detected the installed Tesseract executable:

```
python -c "import pytesseract; pytesseract.pytesseract.tesseract_cmd =
r'C:\Program Files\Tesseract-OCR\tesseract.exe';
print(pytesseract.get_tesseract_version())"
```

Reported version:

```
5.5.3.20260724
```

This confirms that Python can invoke the OCR engine.

10. PDF Extraction Test

File: `backend/test_pdf.py`

A standalone test script verifies the extraction service without involving FastAPI.

The script loads the previously uploaded PDF:

```
from pathlib import Path
from app.services.document_service import extract_pdf_text

pdf_path = (
    Path(__file__).resolve().parent
    / "uploads"
    / "79f96287-7152-4bbe-954a-6bfd3a244cc8.pdf"
)

result = extract_pdf_text(pdf_path)

print("Pages:", result["page_count"])
print("Characters:", result["character_count"])
print("\nExtracted text:\n")
print(result["text"][:1000])
```

Run it from the `backend/` directory:

```
python test_pdf.py
```

Verified test result

```
Pages: 1
Characters: 711

Extracted text:
[Recognized German document text]
```

The test confirmed that the scanned PDF was readable through OCR.

The output included recognizable German headings and document fields, demonstrating that the OCR pipeline can process at least one scanned document successfully.

Some characters were misrecognized. OCR accuracy depends on scan resolution, document layout, image quality, and language configuration.

Deprecation cleanup

The test emitted a warning that the legacy `fitz` API is deprecated.

The recommended import is:

```
import pymupdf
```

Calls such as `fitz.open()` and `fitz.Matrix()` should be changed to `pymupdf.open()` and `pymupdf.Matrix()` respectively.

11. Analysis API

File: `backend/app/routers/analysis.py`

The analysis router connects document upload to the extraction service.

It currently exposes two endpoints.

11.1 Initiate document processing

Endpoint: `POST /analysis/`

Request body

```
{
  "document_id": "79f96287-7152-4bbe-954a-6bfd3a244cc8"
}
```

The endpoint:

1. Validates the document UUID.
2. Finds the uploaded PDF in `backend/uploads/`.
3. Returns an error if the document cannot be found.
4. Calls the document extraction service.
5. Checks that readable text was extracted.
6. Generates a unique analysis ID.
7. Stores the result in an in-memory dictionary.
8. Returns the analysis ID and document-processing statistics.

Verified response

```
{
  "analysis_id": "6ee9cafe-b543-443d-8bab-f0c9ee4e0e46",
  "document_id": "79f96287-7152-4bbe-954a-6bfd3a244cc8",
  "status": "completed",
  "document": {
    "page_count": 1,
    "character_count": 711,
    "ocr_pages": 1
  },
  "message": "Document extraction completed. Compliance comparison is
not implemented yet."
}
```

The endpoint returned HTTP `200` in Swagger UI.

Important: This endpoint currently completes document extraction, not a legal compliance assessment.

11.2 Retrieve analysis results

Endpoint: `GET /analysis/{analysis_id}`

Example:

```
GET /analysis/6ee9cafe-b543-443d-8bab-f0c9ee4e0e46
```

The endpoint validates the UUID and retrieves the stored result.

Possible outcomes include:

- `200 OK`: Analysis found.
- `400 Bad Request`: Invalid analysis ID.
- `404 Not Found`: No result exists for the supplied ID.

The endpoint was tested successfully and returned HTTP `200` with the previously generated analysis result.

Current persistence limitation

Analysis results are stored in an in-memory Python dictionary. They are lost when the backend process restarts.

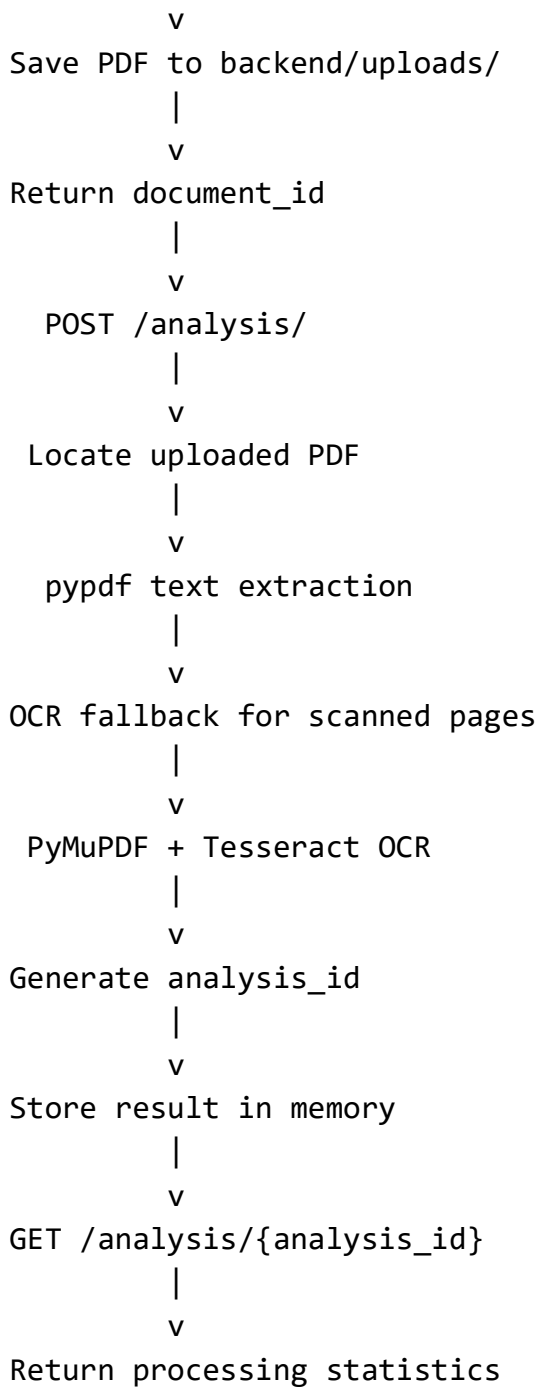
A database-backed repository will be needed for persistent results and analysis history.

12. Current End-to-End Workflow

The current backend workflow is:

```
React / Vite Frontend
          |


          v
     FastAPI Backend
          |
          v
    POST /upload/
          |
```



This pipeline has been exercised successfully using Swagger UI.

13. API Summary

Method	Endpoint	Purpose	Status
GET	`/health/`	Health check	Implemented and tested
POST	`/upload/`	Upload a PDF	Implemented and tested
POST	`/analysis/`	Process an uploaded PDF	Implemented and tested
GET	`/analysis/{analysis_id}`	Retrieve processing results	Implemented and tested

The analysis route is currently named `/analysis/`; it is not yet exposed as `/analyze`.

14. What Is Not Implemented Yet?

The following are planned features, not completed functionality.

Compliance analysis

- Accept a regulation and a company policy together.
- Extract regulatory obligations and policy controls.
- Match requirements against policy evidence.
- Classify coverage as covered, partially covered, or missing.
- Assign risk and severity levels based on defined criteria.
- Produce explainable findings with supporting evidence.

Data and infrastructure

- Persistent database storage.
- Analysis history.
- Background processing for large documents.
- User authentication and authorization.
- Multi-user isolation.
- Production-grade file storage and validation.

AI capabilities

- Semantic matching.
- Embeddings and vector search.
- Retrieval-Augmented Generation (RAG).
- LLM-assisted analysis and explanation.
- Evaluation of accuracy, false positives, and false negatives.

Frontend integration

The frontend and backend are connected, and the backend has been tested through Swagger UI. A complete frontend-driven document-analysis and results-dashboard workflow still needs to be implemented and verified.

15. Next Development Milestone

The next milestone is to implement the actual Policy Gap Analysis Engine.

The planned workflow is:

1. Upload a regulation PDF.
2. Upload a company policy PDF.
3. Extract text from both documents.
4. Identify regulatory obligations.
5. Identify company policy controls and evidence.
6. Compare each obligation against the available policy evidence.
7. Return structured findings with coverage status, rationale, and source references.

8. Display the results in the React dashboard.

The first comparison implementation should be transparent and testable. More advanced semantic matching and LLM capabilities can be introduced after the baseline comparison is validated.

Conclusion

The backend foundation for the Compliance Engine is operational. PDF upload, local storage, text extraction, scanned-document OCR, analysis initiation, and result retrieval have all been tested successfully.

The project is now ready to progress from document processing toward its core purpose: explainable comparison of regulatory obligations against company policies.

Current milestone: Backend document-processing foundation complete; compliance comparison engine pending.